

A Memory-Efficient Hybrid Parallel Framework for Deep Neural Network Training

Dongsheng Li , Shengwei Li , Zhiqian Lai , Yongquan Fu , Xiangyu Ye , Lei Cai , and Linbo Qiao 

Abstract—With the increasing volumes of data samples and deep neural network (DNN) models, efficiently scaling the training of DNN models has become a significant challenge for server clusters with AI accelerators in terms of memory and computing efficiency. Existing parallelism schemes can be broadly classified into three categories: data parallelism (splitting data samples), model parallelism (splitting model parameters), and pipeline model parallelism (splitting model layers). Hybrid approaches split data and models, offering a comprehensive solution for parallel training. However, these methods encounter limitations in efficiently scaling larger models across more computing nodes, as they incur substantial memory constraints that affect training efficiency and overall throughput. In this paper, we propose *HIPPIE*, a hybrid parallel training framework designed to enhance memory efficiency and scalability of large DNN training. First, to evaluate the optimization effect more reasonably, we propose an index of *Memory Efficiency* (ME) to quantify the tradeoff between throughput and memory overhead. Second, driven by the informed ME optimization objective, we automatically partition the pipeline to balance the throughput and memory. Third, we optimize the model training process via a novel hybrid parallel scheduler that improves the throughput and scalability by informed pipeline scheduling and communication scheduling with gradient-hidden optimization. Experiments on various models show that *HIPPIE* achieves above 90% scaling efficiency on a 16-GPU platform. Moreover, *HIPPIE* increases throughput by up to 80%, while saving 57% of memory overhead and achieving $4.18\times$ memory-efficiency improvement.

Index Terms—Data parallelism, pipelined model parallelism, hybrid parallelism, memory efficiency, deep neural network.

I. INTRODUCTION

RECENTLY, larger neural network models have become increasingly popular due to their higher performance. Many of the latest models, such as the GPT series [1], [2], [3] and recommendation systems [4], [5] have billions of parameters, greatly exceeding the memory and computing capability of a single accelerator (e.g., GPU). With the growing memory requirement and computation of the large model training,

applying parallel training on distributed platforms is stringent to accelerate the training process. There are three prevalent parallelism modes, namely, data parallelism (*DP*) [6], [7], [8], model parallelism (*MP*) [9], [10] and pipelined model parallelism (*PMP*) [11], [12], [13], [14], [15], [16], [17].

DP, perhaps the most popular parallelism mode, replicates the full model to each server machine and splits the data samples among these nodes. Training a copy of the entire model with mini-batches sampled from the sharded training data and synchronizing the gradients among all server machines periodically. *DP* is not suitable for large-scale models because of the high memory burden of keeping the full model and the heavy communication overhead with increasing DNN models.

Rather than full model replication in *DP*, *MP* splits models into multiple parts and places them on multiple server machines [9], [10], [18], [19], reducing the memory consumption of model parameters. However, due to the sequential dependencies in the forward and backward propagation, only one server machine could perform computation at any moment. Thus, *MP* has low computational efficiency compared to *DP*. Furthermore, automating the model splitting process in *MP* is challenging due to complex parameter dependencies.

Different from the data dependency in *MP*, *PMP* automates the partition of neural network models by layers and trains these layers in parallel as a pipeline by splitting data samples into separate micro-batches for partitioned models. *PMP* methods can be separated into two categories according to different update rules: asynchronous and synchronous pipelines. One of the state-of-the-art asynchronous pipelines, Pipedream [12], updates model weights as soon as the backward computation is completed to balance throughput and resource utilization but generates multiple model parameter versions. Asynchronous *PMP* such as Pipedream faces high memory requirements because multiple copies of weights will be generated during training. In contrast, synchronous *PMP* [13] updates the model synchronously in grouped micro-batches and reduces memory consumption by avoiding multiple copies of model weights. However, synchronous approaches cause pipeline idles inside each iteration, reducing training efficiency. Although *PMP* scales well for large models in server clusters, it does not fully exploit the server clusters to increase the training throughput, which is vital for large model training. Thus, it is interesting to study how to optimize the integration of *PMP* and *DP*.

Several hybrid parallel approaches [20], [21], [22] seek a comprehensive parallel training scheme. *DAPPLE* [21] partitions models and assigns pipeline stages to different server machines

Manuscript received 1 January 2023; revised 17 October 2023; accepted 10 December 2023. Date of publication 15 December 2023; date of current version 28 February 2024. This work was supported in part by the National Key R&D Program of China under Grant 2021YFB0301200, in part by the National Natural Science Foundation of China under Grants 62025208 and 61972409, and in part by Xiangjiang Laboratory Project under Grant 22XJ01012. Recommended for acceptance by D. Tiwari. (Corresponding author: Zhiqian Lai.)

The authors are with the National Key Laboratory of Parallel and Distributed Computing, College of Computer, National University of Defense Technology, Changsha 410073, China (e-mail: dsli@nudt.edu.cn; swli@nudt.edu.cn; zqlai@nudt.edu.cn; yongquanf@nudt.edu.cn; xyue@nudt.edu.cn; cailei@nudt.edu.cn; qiao.linbo@nudt.edu.cn).

Digital Object Identifier 10.1109/TPDS.2023.3343570

in the cluster, replicating the model stages to different GPU accelerators on the same server machine, and performing *DP* for these accelerators. Different accelerators at the same pipeline location synchronize gradients after each iteration. Although *DAPPLE* relieves memory pressure to a certain extent by adjusting the micro-batch calculation order, it faces scaling bounds for large server clusters, as the number of pipeline stages puts the upper bound on the scale of participating server machines. Therefore, it is desirable to simultaneously meet the requirements of efficient model partitioning, low memory overhead, and high scalability.

In this paper, we introduce *HIPPIE*, a hybrid parallel framework designed to automatically partition models, maintain high throughput, reduce the memory overhead for large-scale DNN models, and ensure scalability in training on large-scale clusters. *HIPPIE* utilizes optimized *PMP* with *DP* support for server clusters.

First, for a more reasonable evaluation of the performance of the pipelines and data-parallelism model training, we present a memory-efficiency (ME) index including two vital metrics, throughput, and memory overhead, to measure the efficiency of accelerators.

Second, based on the informed ME index, we present *HIPPIE Planner* for balanced and efficient training, which comprehensively considers various factors in training and automatically yields model partitions with automatic checkpoints [23], [24].

Third, *HIPPIE* hides the gradient communication, optimizes memory overhead, and partitions the model to address problems in existing hybrid parallel methods. We feature a novel communication Schedule to hide the gradient communication and reduce idle time, improving scalability and facilitating distributed training in large clusters. For the convenience of expansion, *HIPPIE* forms pipeline machine groups and conducts training on multiple nodes.

HIPPIE is implemented using PyTorch and NCCL and we tested *HIPPIE* on various benchmarks. Experiments on various models show that *HIPPIE* can improve scaling efficiency, increase throughput, and save memory significantly. Besides, *HIPPIE Planner* can achieve the best model partition schemes among the three methods. Moreover, *HIPPIE* can improve the memory-efficiency index than *DP* and achieve better scaling efficiency than *DAPPLE* while maintaining the model convergence.

II. BACKGROUND AND MOTIVATIONS

A. Pipeline and Data-Parallelism Based Model Training

For ease of presentation, we consider the parallel model training with a homogeneous GPU cluster with a high-speed network such as the Infiniband, where each server machine is configured with several GPU devices.

Data parallelism: Data parallelism is one of the most widely adopted model training paradigms atop the GPU cluster. A server machine set performs the data-parallel DNN training process based on collective communication, such as AllReduce, by directly communicating with each other in synchronous iterations to reduce the gradients and update the DNN weights.

Pipeline model parallelism: The pipeline model training differs from data parallelism, which partitions the model into several stages, each consisting of a sequence of neural network layers. These stages are trained on GPU devices in iterations. During an iteration, the accelerators mainly have five states: forward computation (also called inference, the process of performing output activations based on the input and weights), backward computation (also called back-propagation, the process of finding the gradients for weights and the input to a specific loss function using the output from the forward computation), gradient communication (exchanging gradients within the pipeline and between distributed server machines), parameters update (update the gradients and weights), and idle status (due to the delay of waiting for the dependent process to be completed as well as the network communication delay). To adapt to pipelined training of stages, the sample batch for the model training is further split into several micro-batches that are continuously fed to the stages in parallel since micro-batches do not have dependence on each other.

Model partitioning: Pipeline model parallel training requires the partitioning of models. Existing model partition methods share a common optimization objective of maximizing training throughput and can be categorized into two primary approaches: i) heuristic rule-based methods [25], which partition models based on experiential rules; and ii) algorithm-based methods [12], [21], [26], which employ objective functions to search an optimal partitioning strategy.

B. Limitations of Prior Approaches

A unified optimization goal: Prior pipeline-based parallel training approaches need a more unified optimization goal to balance the requirements of training throughput and memory. For instance, Pipedream increases throughput and memory footprint, while Gpipe reduces memory overhead and decreases throughput. *DAPPLE* [21], which combines Gpipe and *DP*, is considered one of the state-of-the-art pipeline approaches. Fig. 1(a) shows the communication topology of *DAPPLE*, it performs *DP* for intra-nodes and *PMP* for inter-nodes. *DAPPLE* faces the scalability issue since the number of pipeline stages determines the maximal number of participating machines. Therefore, the first core question of this paper is *how to design an index for joint optimization of throughput, memory, and scalability*.

A comprehensive partition method: *PMP* performs well on fewer nodes, and *DP* makes for easy scaling. To combine the advantages of integrating *PMP* and *DP* and avoid scalability issues in *DAPPLE*, *HIPPIE* use a hybrid parallelism method as shown in Fig. 1(b). Existing model partition methods for *PMP* [12], [21] focus on maximizing throughput on a given training configuration; users need to acknowledge whether this training configuration can fully utilize a training cluster. So, we need to consider memory, making sure that we're able to use all training resources. The second core problem of this paper to solve is *how to partition model atop a unified optimization goal*.

An efficient pipeline scheduler: During the training of neural network models, we employ data parallelism among pipeline

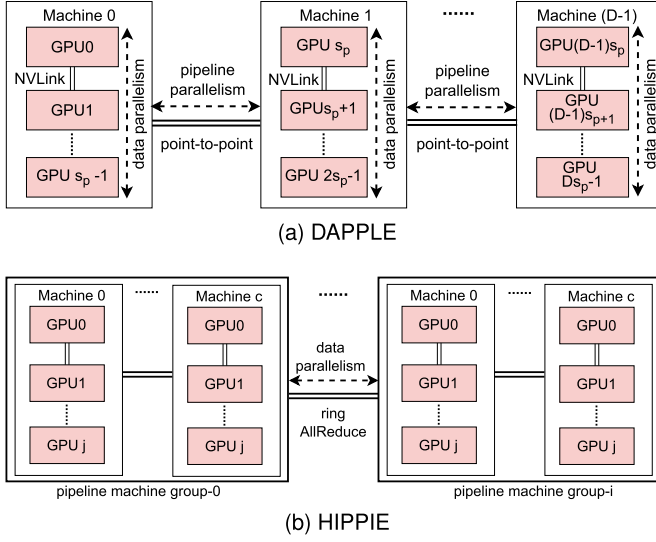


Fig. 1. Illustrative example of the communication topology of *DAPPLE* and *HIPPIE*. Machines in the same group are called intra-nodes. Machines in different groups are called inter-nodes.

machine groups. This process requires collective communication, such as AllReduce, to exchange meta-data for gradient averaging to maintain synchronized neural network models. A naive gradient-communication approach is to perform the AllReduce after an iteration of backward computation, which delays the next pipeline iteration. And idle pipeline “bubbles” resulting from synchronized gradient communication or the completion of forward and backward operations persist. Thus, to increase the pipeline’s throughput, another stringent problem this paper wants to answer is *how to reduce the gradient-communication delay and the pipeline idle bubbles*.

III. HIPPIE OVERVIEW

A. Memory-Efficiency Index

Optimizing throughput alone tends to overuse device memory, which may reduce the efficiency of device memory utilization per unit, and users may misjudge the overall efficiency of the cluster. Moreover, large-scale training is usually bounded by the memory of devices. To this end, we need to reduce memory overhead while maintaining high throughput. We propose the *Memory Efficiency (ME)*, which is formally defined as follows:

$$ME = \frac{\text{Scalability} \cdot \text{Throughput}}{\text{Memory}}. \quad (1)$$

The ME index measures the overall efficiency of the parallel approach through the ratio of the average throughput per device after expansion to maximum memory consumption per device (GB). With the ME index, our research can focus on three aspects simultaneously: improving scalability, reducing memory footprint, and balancing the partitioned model.

The optimization goal in (1) considers both throughput and memory footprint. We can rewrite ME as follows:

$$ME = \frac{X}{\Omega_{\max}}, \quad (2)$$

where the memory overhead $\Omega_{\max}$ (in GB) denotes the maximal memory usage among all GPU devices, and the service rate X (in samples/second) denotes the average rate at which the training process completes an iteration.

Let T_{exec} (in seconds) denote the expected service time in an iteration. The service rate X is inversely proportional to the expected service time in an iteration, so we can use $1/T_{exec}$ to represent the service rate. Then the memory-efficiency index ME in (2) can be expressed as:

$$ME = \frac{1}{T_{exec} \cdot \Omega_{\max}}. \quad (3)$$

We define a cost function h_{ME} with respect to (3) as:

$$h_{ME} = T_{exec} \cdot \Omega_{\max}. \quad (4)$$

Our optimization goal equivalently seeks to minimize the cost function h_{ME} . Our primary goal is to increase the memory-efficiency index for deep neural network training on GPU clusters.

B. The HIPPIE Architecture

We present a novel framework *HIPPIE* to automatically partition the DNN model into pipelines and orchestrate the distributed training atop the PyTorch environment.

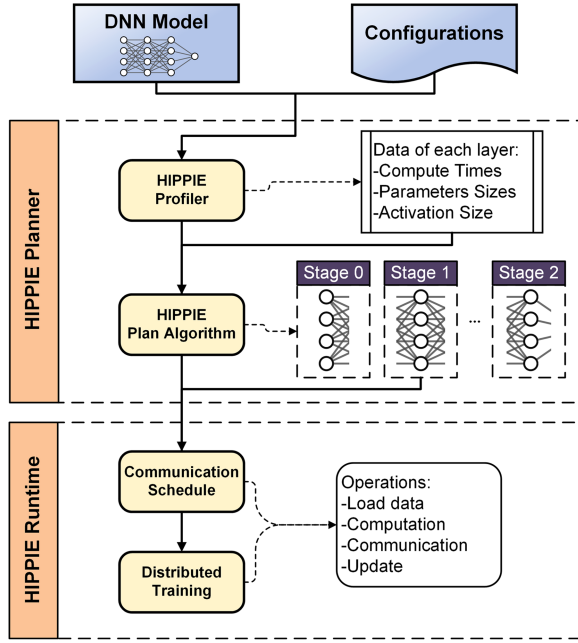
HIPPIE applies pipelined model training based on a hybrid parallel framework, where several computing machines are organized into a pipeline machine group, which performs pipeline computation for GPU devices in the same group. While for different groups of machines, *HIPPIE* applies the data-parallelism training for collective gradient sharing among different pipeline machine groups.

HIPPIE consists of *HIPPIE Planner* and *HIPPIE Runtime*, as shown in Fig. 2. Given a model and a training configuration, the *HIPPIE Planner* adaptively partitions the model and selects the layers needing a checkpoint based on the *HIPPIE Profiler*. The profiler employs simulated training experiences to record the runtime performance parameters of each layer (e.g. computing period, parameter sizes, and activation sizes) and produces a model profiling set. Subsequently, a balanced pipeline plan is automatically constructed, which can be readily expanded to more nodes by duplicating itself.

HIPPIE Runtime supports the distributed training process while preserving the order of the pipeline and the efficiency of *HIPPIE* training operations.

C. HIPPIE Techniques

Scheduling the execution of stages and arranging them reasonably are essential for improving throughput and scalability. An iteration’s process can be divided into three phases: Warmup (filling up the pipeline), Steady (running the nearly-full pipeline), and Ending (finishing the remaining stages of the

Fig. 2. High-level workflow of *HIPPIE*.

pipeline). Recently, Beaumont et al. [27] demonstrated that the well-known one-forward-one-backward (denoted as *1F1B* for short) rule [12] is optimal for memory usage. Therefore, in this paper, we adhere to the *1F1B* rule in the steady phase, where each stage initiates the backward computation as soon as possible, ensuring the relative order of micro-batches between the forward and backward stages is preserved. Let the number of stages in the pipeline be s_p . The execution sequences of different stages may vary. Yet, completing a micro-batch still requires $2s_p$ units of time, i.e., from the start of the forward computing of this micro-batch to the end of the backward computation of this micro-batch.

First, *HIPPIE* avoids the performance degradation caused by over-partitioning of the models by limiting the pipelined training to a fixed number of GPU devices in the same machine group. Due to the dynamic organization of the pipeline machine group, *HIPPIE* adapts to a varying scale of deep learning modeling requirements. For instance, a single machine equipped with several GPU devices can be used as a pipeline machine group, especially for small to medium-sized models. In this group, GPU devices in the same machine conduct pipeline computing. However, for extremely large models that cannot fit on a single machine, *HIPPIE* partitions the model by layers and distributes them to different GPU devices in multiple machines. Then, GPU devices perform pipeline computing and exchange data among machines in the same group.

Second, *HIPPIE* scales the collective communication among distributed machines via the data-parallelism training, which adapts to the trend of large-scale distributed machines connected with high-speed network topology. Since it smoothly adds more machines via the ring-AllReduce collective communication support.

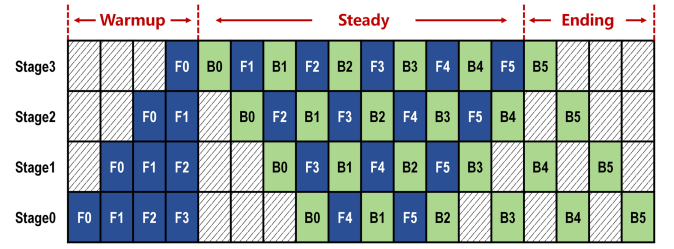


Fig. 3. There are three phases (Warmup, Steady, and Ending) and four pipeline stages vertically in an iteration of the pipeline. We use blue and green blocks to indicate forward and backward computation and yellow blocks to indicate gradient communication and updates. The numbers in cells represent the micro-batch IDs within an iteration.

Third, *HIPPIE* reduces the pipeline idle period (“bubble”) via a communication scheduling technique to hide the synchronized collective communication for the data-parallelism model training. Although the pipeline iteration maintains the order between pipeline stages, it significantly reduces the chances of idles caused by waiting for collective communication input. The communication scheduling not only reduces the overall time taken to train the models in the pipeline, but also improves the efficiency of the training process.

IV. *HIPPIE* PLANNER

After providing an overview of *HIPPIE*, we will introduce the *HIPPIE* planner, which comes with a sophisticated profiling model to measure the memory efficiency index. This index is a crucial metric for evaluating the pipeline planning process. We will profile the memory efficiency index for the pipelined parallel environment and then present the model partitioning and mapping of pipelines to the topology and communication groups.

A. Memory-Efficiency Index h_{ME} Profiler

We will first present our process for profiling the memory-efficiency index h_{ME} , which is crucial for partitioning neural network models.

Example 1: An exemplary pipeline in an iteration is shown in Fig. 3. There are six groups of micro-batches processed atop four pipeline stages. Stage3 serves as the last stage of the pipeline, which is the terminating point for forwarding computing and the starting point for backward computing. Thus, Stage3 has the shortest time interval between forwarding and backward computing. At the same time, Stage0 is the last stage to perform backward computing, which needs multiple rounds of forward computing and temporally keeping intermediate data. The volume of the temporary data kept for each stage is proportional to the relative order of this stage in the pipeline. For example, Stage0 needs to keep four groups of forward-computing intermediate results, while Stage1 keeps three groups of forward-computing intermediate results, and so on.

For each phase (Warmup, Steady, and Ending as shown in Fig. 3) in an iteration, both memory consumption Ω_s and time consumption T is calculated. The Ω_s contains model parameters,

optimizer parameters, and activation parameters. In particular, the accumulation of activation is mainly completed in the warmup phase, and the activation data is constant during the steady phase.

Our analysis involves a set of layerwise parameters as well as a set of global parameters. (i) The global parameters include: s_p , the number of stages in the pipeline; N , the number of micro-batches in one iteration, satisfying that $N \geq s_p$; n , the number of micro-batches; and L_s , the number of layers in the s -th stage. (ii) The layer-wise input for the l -th layer in the model include: this layer's parameter W_l , this layer's activation parameter τ_l , the ratio γ of the memory cost to the size of the parameter W_l for layer l , the forward time of a micro-batch T_F^l , the backward time of a micro-batch T_B^l .

1) *Memory*: The memory cost includes the storage of the weight parameters and the activation parameters.

Let us consider the s -th stage of a s_p -stage pipeline. First, the memory size of the weight parameters is linear to the size of the weight parameters, which can be presented by γW_l for each layer l . Second, the memory size of the activation parameters. Due to the one-forward-one-backward (1F1B) rule in the steady phase, the number of activation data no longer changes. Thus, we can calculate the memory consumption when the activation parameters reach the maximum, the junction point between the warmup and steady phase.

For the first stage, the backward computing passes the results after $2 \cdot s_p$ time, which needs to keep s_p groups of activation parameters. For the last stage, its backward computing follows its forward computing, thus it just needs one group of activation parameters. By induction, the s -th stage needs to keep $(s_p - s)$ groups of activation parameters. In summary, the total memory consumption of the s -th stage can be expressed as the following formula:

$$Mem = \max_{s=0}^{s_p-1} \Omega_s, \text{ where, } \Omega_s = \gamma \cdot \sum_{l=0}^{L_s} W_l + (s_p - s) \cdot \sum_{l=0}^{L_s} \tau_l. \quad (5)$$

HIPPIE employs checkpointing to reduce memory usage, leading to the discarding of activation values during forward computation. When the model performs backward computations and requires specific operator activation, the forward computation is re-executed. Consequently, when modeling h_{ME} , checkpointing influences memory overhead and total processing time. This technique exchanges the doubling of forward time for reduced activation, making it particularly suitable for layers with substantial activation parameters and rapid forward computations. *HIPPIE* selects layers for checkpointing based on the following criteria: activation accounting for more than 15% of the total activation and forward time contributing less than 20% of the total forward time. After applying the checkpoint, there are slight changes in the layer's statistical data. The backward time T_B^l increases by T_F^l , and the activation becomes $\tau_l = 0$, while forward time and parameter size remain unchanged.

2) *Time*: Next, we calculate the total time for an iteration. During the warmup phase, forward computations in each stage start sequentially due to data dependencies between layers.

Therefore, the warmup time T_1 is the sum of the forward times for each stage. Since the forward time in the s -th stage is represented as $\sum_{l=0}^{L_s} T_F^l$, time T_1 can be summarized as follows:

$$T_1 = \sum_{l=0}^{L_0} T_F^l + \sum_{l=0}^{L_1} T_F^l + \cdots + \sum_{l=0}^{L_{s_p-1}} T_F^l = \sum_{s=0}^{s_p-1} \sum_{l=0}^{L_s} T_F^l. \quad (6)$$

In the steady phase, it is necessary to identify the stage with the longest sum of forward and backward times since all stages must wait for the slowest stage, which determines the overall computation time. Assuming that the s_{slow} -th stage takes the longest time, the following conditions must be met:

$$T_{s_{slow}} = \sum_{l=s_{slow}}^{L_{s_{slow}}} (T_F^l + T_B^l) = \max_{s=0}^{s_p-1} \sum_{l=0}^{L_s} (T_F^l + T_B^l).$$

The backward computation starts at the beginning of the last steady phase, the calculation of the steady phase time can be divided into three parts:

- The backward process from the first micro-batch in the last stage to the s_{slow} -th stage.
- The process from the first micro-batch to the last micro-batch in s_{slow} -th stage, with the s_{slow} -th stage determining the time for this part.
- The forward process from the s_{slow} -th stage to the last stage, with the time being the sum of the forward times for all the stages along the way.

Example 2: Let us consider Fig. 3 as an example. Assuming that Stage1 is the longest stage, the calculating of the steady phase only involves Stage1, Stage2, and Stage3. This analysis is orthogonal to Stage0. Then, given that the s_{slow} -th stage takes the longest time and the number of micro-batches is N , the steady phase time T_2 can be expressed by the following formula:

$$T_2 = \sum_{s=s_{slow}+1}^{s_p-1} \sum_{l=0}^{L_s} (T_F^l + T_B^l) + (N - 1 - (s_p - s_{slow})) \cdot \sum_{l=0}^{L_{s_{slow}}} (T_F^l + T_B^l). \quad (7)$$

The time T_3 of the ending phase is similar to that of the warm-up phase, with each stage performing backward in turn. Therefore, the time of the ending phase is expressed as:

$$T_3 = \sum_{l=0}^{L_0} T_B^l + \sum_{l=0}^{L_1} T_B^l + \cdots + \sum_{l=0}^{L_{s_p-1}} T_B^l = \sum_{s=0}^{s_p-1} \sum_{l=0}^{L_s} T_B^l. \quad (8)$$

Combining (6), (7), (8), the total time T of an iteration is the sum of the time used by the three phases: $T = T_1 + T_2 + T_3$. By induction, the specific expression for a pipeline with s_p stages is as follows:

$$T = \sum_{s=0}^{s_p-1} \sum_{l=0}^{L_s} (T_F^l + T_B^l) + \sum_{s=s_{slow}+1}^{s_p-1} \sum_{l=0}^{L_s} (T_F^l + T_B^l) + (N - 1 - (s_p - s_{slow})) \cdot \sum_{l=0}^{L_{s_{slow}}} (T_F^l + T_B^l). \quad (9)$$

3) *Overall Metric*: The final optimization goal h_{ME} is calculated by the (5) and the (9). The memory consumption is calculated with (5).

$$\begin{aligned}
 h_{ME} &= T \cdot \max_{s=0}^{s_p-1} \Omega_s \\
 &= \left(\sum_{s=0}^{s_p-1} \sum_{l=0}^{L_s} (T_F^l + T_B^l) + \sum_{s=s_{slow}+1}^{s_p-1} \sum_{l=0}^{L_s} (T_F^l + T_B^l) \right. \\
 &\quad \left. + (N-1-(s_p-s_{slow})) \cdot \sum_{l=0}^{L_{slow}} (T_F^l + T_B^l) \right) \\
 &\quad \cdot \max_{s=0}^{s_p-1} \left(\gamma \cdot \sum_{l=0}^{L_s} W_l + (s_p-s) \cdot \sum_{l=0}^{L_s} \tau_l \right). \quad (10)
 \end{aligned}$$

As shown in (10), the number of micro-batches, denoted as N , and the partition position of the model together determine the cost function h_{ME} .

4) *Parameter Profiling*: After presenting the equation of the memory-efficiency index, we proceed to concretize the parameters in (10) to estimate the memory-efficiency index for a specific pipeline model. To achieve this, we employ simulated training to acquire these parameters. First, we partition each layer within the neural network model, treating each individual layer as a separately running neural network with identical data dependencies. Second, we provide simulated data samples for training the layered neural network models. The forward computation follows the sequential orders of these layers in the original model, while the backward computation proceeds in reverse order. Throughout the simulated runs, we record the time taken for both forward and backward computations for each layer, the memory consumption during the computation process, and the sizes of the parameter and activation data. We subsequently compile these parameters into a comprehensive table, which serves as the profiling results.

B. Partitioning of Neural Network Models

Next, we introduce the partitioning framework for neural network models, referred to as *HIPPIE*, which directly minimizes the memory-efficiency optimization objective to obtain the partitioned model of the neural network.

The objective is to find the most efficient solution that minimizes the memory-efficiency index h_{ME} over various combinations of model partitions across different stages. With a fixed global batch size and ensuring each stage contains at least one layer, where the global batch size is divisible by N , the *HIPPIE* Planner automatically partitions the DNN model and applies checkpointing to ensure memory efficiency during training. This partitioning is determined based on statistical data for each layer provided by the *HIPPIE* Profiler.

The input for the partitioning process is the memory-efficiency cost h_{ME} , which is derived from the *HIPPIE* Profiler of the neural network. Optimizing the memory-efficiency cost results in a search space comprising all possible combinations of micro-batch numbers and layer distributions within each stage. This search space is independent of the parameter profiling

Algorithm 1: Partitioning Process of the DNN Model for a Four-Stage ($s_p = 4$) Pipeline. We Represent the Number of Model Layers for Each Stage as a Tuple (a, b, c, d), Where Stages are Implicitly Indexed in Ascending Order in the Tuple.

Input: layer L , batch size B , micro-batch size N .

Output: Plan output: (a, b, c, d).

Initialize: Pipeline stages $S = 4$, Cost function h_{ME} .

```

1: function PLAN( $L, B, N$ )
2:   (a, b, c, d)  $\leftarrow$  zeros
3:    $F \leftarrow +\infty$ 
4:   for all  $s_1 = 1$  to  $L - 3$ 
5:     for all  $s_2 = s_1 + 1$  to  $L - 2$ 
6:       for all  $s_3 = s_2 + 1$  to  $L - 1$ 
7:         for all  $s_4 = s_3 + 1$  to  $L$ 
8:            $f \leftarrow h_{ME}(s_1, s_2, s_3, s_4, B, N, S)$ 
9:           if  $f < F$  then
10:             $F = f$ 
11:            (a, b, c, d)  $\leftarrow$  ( $s_1, s_2, s_3, s_4$ )
12:           end if
13:         end for
14:       end for
15:     end for
16:   end for
17: end Function

```

process, as the profiling2 table is obtained through simulated runs. Algorithm 1 summarizes the partitioning process for a four-stage pipeline.

C. Topology Mapping

Based on the partitioned model, *HIPPIE* maps the model layers to the stage. Each pipeline corresponds to a separate pipeline machine group denoted as a virtual *worker* for a fixed number of server machines in the GPU cluster. Scaling out *HIPPIE* by replicating virtual workers to more distributed nodes is convenient.

The communication cost for intra-pipeline devices is modest in scale but relatively high frequency due to the pipelined operations. The communication cost for inter-pipeline devices involves gradient communication, which is large in scale but in relatively low frequency due to the synchronization control. Therefore, we employ a heuristic rule to finalize the topology mapping. Specifically, let $(d_0, d_1, \dots, d_i, \dots)$ denote the device set of a training cluster and S become the number of pipeline stages, when the inter-pipeline communication volume is greater than the intra-pipeline, and the inter-pipeline group size is smaller than the number of devices on each machine, we group the stages within the same inter-pipeline on a single machine, which is, the device d_i will hold the pipeline stage $\lfloor i/S \rfloor$. This ensures that gradient communication occurs within the intra-node. Otherwise, we organize the stages within the same intra-pipeline groups on adjacent machines, which is, the device d_i will hold the pipeline stage $i \bmod S$.

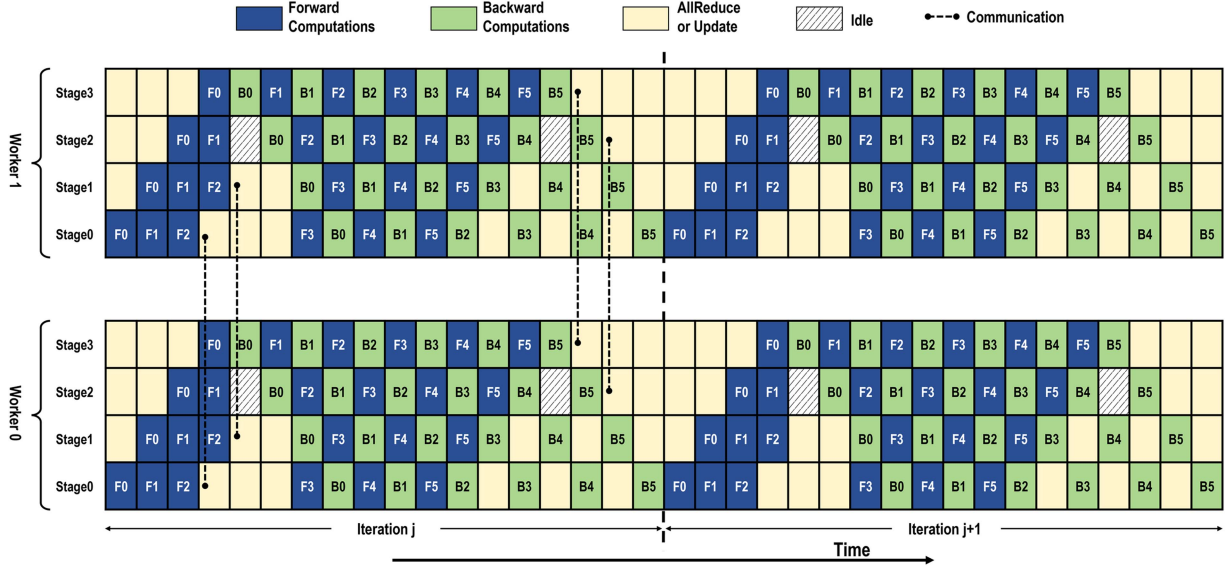


Fig. 4. Illustrative example of *HIPPIE* based pipeline. We use a worker to refer to the machines in the same pipeline group for simplicity. We use blue and green blocks to indicate forward and backward computation and yellow blocks to indicate gradient communication and updates. The numbers in cells represent the micro-batch IDs within an iteration.

V. PIPELINE SCHEDULING

Having presented the planner framework to optimize the memory-efficiency index, we next present scheduling for *HIPPIE*.

A. Informed Pipeline Scheduling

The main operations of each stage in *HIPPIE* include forward computation, backward computation, intra-pipeline communication (sending data, gradients, and weights between pipeline stages), and inter-pipeline communication (exchanging data, gradients, and weights between pipelines).

Example 3: As shown in Fig. 4, each intra-worker stage cooperates with adjacent stages by transferring intermediate data of layers. At the same time, stages of the same location in workers synchronize gradient through AllReduce. The proportion of communication time and idle time should be reduced as much as possible. *HIPPIE* advances communication operations and covers it within the computation process.

Through the example 3, one key observation is that *HIPPIE* splits each micro-batch's forward and backward computation to multiple processes and spawned to several devices; the timeline of training is usually presented as a trapezoidal shape. Thus, we can change the sequential order to a parallel order by splitting the gradient communication into several sub-procedures and putting these sub-procedures into idle pipeline bubbles: (i) For the stages in the first half of the pipeline (denoted as stage-up), the idle time within an iteration is the longest. Thus, we rearrange the communication and averaging of gradients to fill these pipeline bubbles. (ii) For stages in the second half of the pipeline (denoted as stage-down), the longest idle time is located between two iterations. Therefore, the synchronization of their gradients can be done without rearrangements.

We introduce our communication schedule to change the relative order of computation and communication, which hides all the operations of gradient communication during the computation process. As shown in Fig. 4, there is almost no delay between each iteration, which improves the pipeline's scalability.

The parameter update rules in *HIPPIE* can be summarized as synchronous intra-iteration updates and asynchronous inter-iteration updates. Synchronous intra-iteration update means that the micro-batch updates within an iteration are kept in sync to ensure that multiple model weights will not be created. The asynchronous inter-iteration update specifically refers to the update strategy for the second half of the pipeline. *HIPPIE* updates weights with the averaged gradients of the previous iterations in these stages.

Algorithm 2 summarizes key scheduling steps for these two classes of stages. As the gradient communication in stage-up advances, one more operation for exchanging gradient is needed before the update. Specifically, the newly calculated gradients are exchanged with the averaged gradients. The averaged gradients are used for updating, while the new gradients are used for AllReduce in the next iteration.

B. Scheduling Analysis

We next describe the analysis of the communication schedule in *HIPPIE*.

1) Convergence: We first show that the asynchronous communication does not break the convergence of the overall process inspired by [28]. Let w_l denote the stages' weights in the pipeline's first half that perform synchronous updates. Let w_f for weights of stages in the second half of the pipeline that perform asynchronous updates. Let w_{l-init} denote the initial values of

Algorithm 2: Stages Execution With Communication Schedule.

```

1: function STAGESCHEDULER(stage  $s$ )
2:   if  $s$  located at the first half of the pipeline and
     applied Communication Schedule then
3:      $(s_p - s)$  micro-batches to forward computation;
4:      $(n - (s_p - s))$  micro-batches to forward and
       backward computation;
5:      $(s_p - s)$  micro-batches to backward computation;
6:     Synchronization via the AllReduce;
7:     Updating local gradients;
8:   else
9:      $(s_p - s)$  micro-batches to forward computation;
10:    Asynchronous communication of gradients in the
      buffer;
11:     $(n - (s_p - s))$  micro-batches to forward and
      backward computation;
12:     $(s_p - s)$  micro-batches to backward computation;
13:    Wait until the asynchronous communication is
      completed;
14:    Swapping the new gradients with the gradients in
      the buffer;
15:    Updating local gradients;
16:   end if
17: end Function

```

w_l . Let the gradients calculated by the t -th global batch be expressed by $g^{(t)}$.

(i) Under a synchronous updating scheme, the weight parameter w_l after n micro-batches can be expressed as follows:

$$w_l = w_{l-init} - v \cdot (g^{(1)} - g^{(2)} - g^{(3)} - \dots - g^{(t)}).$$

(ii) Under an asynchronous updating scheme, the weight parameter w_f can be expressed as follows:

$$w_f = w_{f-init} - v \cdot (0 - g^{(1)} - g^{(2)} - \dots - g^{(t-1)}).$$

(iii) Findings: We see that compared with the synchronous weight training parameter w_l , the asynchronous weight training parameter w_f does not update the parameter at the first communication, and the recursive gradient of w_f at the last update is $g^{(t-1)}$, which is the last by one global batch for w_l .

Although w_l lacks one global-batch update compared to w_f , the training paths are the same. Since we can equivalently represent w_{l-init} by $(w_{l-init} - v \cdot g^{(1)})$, then the update path of w_l will be the same with that of w_f . Thus, asynchronous communication still converges, similar to synchronous communication-based training.

2) *Idle Time:* We next compare the device's idle time with the time spent in AllReduce operations and analyze the feasibility of using idle time to hide the communication overhead completely. The global parameters include s_p , the number of stages in the pipeline; N , the number of micro-batches in one iteration, satisfying that $N \geq s_p$; and L_s , the number of layers in the s -th stage.

(i) For *DP*, assume that the number of stages in the hybrid parallel training scheme is s_p , and the number of micro-batches is N . Under ideal conditions, the model parameters are divided equally. Then, the data communicated by the AllReduce operation in each stage is the same, which is $1/s_p$ of the *DP*. Meanwhile, compared with *DP*, the number of nodes that need to communicate becomes $1/s_p$ due to the s_p stages. Then, the communication volume of AllReduce operations in a stage is $1/s_p^2$ of *DP*.

Let the computational time in *DP* be denoted by $T_{minibatch}$. Assuming that the ratio of communication overhead to computational overhead is r_{cc} . The communication time in a stage can be expressed as:

$$T_{com} = \frac{1}{s_p^2} \cdot r_{cc} \cdot T_{minibatch}. \quad (11)$$

If the calculation time is proportional to the batch size, a mini-batch will be divided into N micro-batches. Then the calculation time of a micro-batch will be $1/(s_p \cdot N) \cdot T_{minibatch}$.

(ii) For *HIPPIE*, the shortest idle time available for AllReduce operations is the computation of $s_p/2$ micro-batches, which is expressed as:

$$T_{idle} = \frac{1}{s_p \cdot N} \cdot \frac{s_p}{2} \cdot T_{minibatch} = \frac{1}{2 \cdot N} \cdot T_{minibatch}. \quad (12)$$

(iii) Findings: Combining (11) and (12), if $T_{com} \leq T_{idle}$, the communication overhead can be hidden completely, which requires that the condition $r_{cc} \leq s_p^2/(2 \cdot N)$ needs to be satisfied.

The number N of micro-batches is typically greater than the number s_p of stages. In practice, we set $N = 4 \times s_p$. When $N = s_p$, the condition above becomes $r_{cc} \leq s_p/2$. Furthermore, assuming that $N = 2 \cdot s_p$, the condition becomes $r_{cc} \leq s_p/4$, which means $r_{cc} \leq 1$ for the 4-stage pipeline.

In *DP*, it is easy to meet the communication-computation ratio r_{cc} of less than 1. Therefore, the communication schedule in *HIPPIE* has a high degree of tolerance for communication overhead and can completely hide it in many cases.

VI. RUNTIME ORCHESTRATION

Communication implementation: Before training, *HIPPIE* performs a grouping of stages for synchronous and asynchronous communication. Each stage is given two identities: a stage identifier in the pipeline and a worker identifier in the data parallelism. The inter-worker stages are then grouped to form an inter-group for gradient averaging.

First, each stage groups with neighboring stages to transfer forward computation results and gradients named intra-group. We express intra-group communication and computation as a whole, called a single forward or backward.

Second, the stages at the same pipeline position are divided into inter-groups. Inter-group communication is performed via the AllReduce-based collective communication. Inter-group gradients will be averaged to maintain synchronization through AllReduce.

Stage runtime management: First, only the first stage (denoted as stage0) of all the stages in the pipeline needs to inject new data into the pipeline. The training data for the rest stages are

all obtained by communication. Second, during training, the outputs of the local forward computation are stored in GPU memory. These outputs will be released once the micro-batch completes its backward pass.

According to the chain rule in back-propagation, the gradients of inputs have to be taken out and transmitted to the previous stage to continue backward computation. In PyTorch, it does not save gradients by default.

To support the communication scheduling process, we explicitly register hook functions by putting the intra-group communication function into the hook, so the gradients will be immediately passed to the next as soon as the local backward computations are completed. Further, all intermediate gradients are stored in a gradient buffer. Gradients in the buffer are still in the dictionary order. Therefore, during AllReduce operations, the correspondence of data dimension can be guaranteed. After each iteration, the newly calculated gradients will be exchanged with the data in buffers so that averaged gradients are applied to update the weights.

Boundary data: Boundary data is the input and output data of each pipeline stage. According to the chain rule in back-propagation, the gradients of inputs have to be taken out and transmitted to the previous stage to continue backward computations. In PyTorch, leaf data in a graph, such as the input, do not save gradients by default, which needed to explicitly define that it needs gradient by registering hook. For stages excluding the last, the gradients obtained by the hook will be immediately passed to the next as soon as the local backward computations are completed. For the last stage, transmissions start from half of the step. The outputs of the local forward computations are stored in GPU memory, and its pointer is put in the work queue. These outputs will not be released until the mini-batch completes its backward pass.

VII. EVALUATION

In this section, we demonstrate that *HIPPLE* could effectively solve the problems we observed in Section II-B as follows:

- To assess the validity of *HIPPLE* pipeline scheduling, we employ identical partition methods from our planner. We conduct a comparative analysis of the end-to-end training performance of *HIPPLE* when compared with state-of-the-art approaches (Section VII-B).
- To show the effectiveness of *HIPPLE* planner, we compare it with the two additional rule-based partitioning methods integrated into our pipeline scheduling. The comparison encompasses an evaluation of memory overhead, throughput, and memory efficiency (Section VII-C).
- In order to demonstrate the scalability of *HIPPLE*, we scale the training resource and compare the training performance of *HIPPLE* with *DAPPLE* and ideal linear scaling outcomes. (Section VII-D)

A. Experimental Setup

Models and datasets: We test the performance of *HIPPLE* among four diverse DNN models, including image classification

TABLE I
MODELS, DATASETS, AND CORRESPONDING ACCURACY TARGET USED IN OUR EXPERIMENTS

Model	# of Params	Dataset	Target Accuracy
GNMT-8	191M	WMT16 EN-De	24 BLEU
GNMT-16	290M	WMT16 EN-De	24 BLEU
VGG-16	138M	ImageNet	70% top-1
AmoebaNet-18	318M	ImageNet	70% top-1

and NLP tasks. As shown in Table I, we train the models of VGG-16 [29] and AmoebaNet(18, 416) [30] with the ImageNet [31], GNMT-8 and GNMT-16 [32] with the WMT16 En-De [33]. The AmoebaNet (L, D) denotes the AmoebaNet-D model with L normal cell layers and filter size D, and GNMT-N represents the GNMT model containing N LSTM layers.

Hardware configuration: Our experiments are based on a GPU cluster, using up to 4 server machines. Every node is equipped with four NVIDIA GeForce RTX 3090 GPUs with 24 GB memory and two Intel Xeon 4214R @ 2.40 GHz CPUs. All servers run 64-bit Ubuntu 18.04, CUDA 11.1, cuDNN 8.0.5, NCCL 2.7.8 [34], PyTorch 1.8, and are connected by 100 Gbps InfiniBand.

Batch sizes: We test various global batch sizes (GBS) to explore the GPU memory overhead, throughput, and scalability of different approaches. The computation time of models varies from batch size, while the communication traffic remains the same. In experiments of model partitioning strategy, the *HIPPLE* training scheme is adopted with various GBS. Due to the limitation of memory overhead in *DP*, the setting of GBS has some limitations. For VGG and GNMT, there are four GBS groups of 256, 512, 1024, and 2048, while GBS 32, 64, 128, and 256 for AmoebaNet require larger memory. Consequently, we can track each method's memory-efficiency index and scalability with multifarious computation communication ratios to test the performance.

Hyperparameters: The choice of hyperparameters is consistent with state-of-the-art practices of the ML community. We applied the Adam optimizer to train GNMT and the SGD optimizer for VGG and AmoebaNet. The initial learning rates for GNMT-8, GNMT-16, VGG-16, and AmoebaNet-18 are 0.001, 0.001, 0.01, and 0.1, respectively. The target validation accuracy is as follows: top-1 accuracy of 70% for image classification and BLEU score of 24 for GNMT. During the training epochs, we properly adjusted the learning rate according to prior works and applied the learning rate warmup to accelerate the convergence in the demands of models. All the training methods in the same experiment set used the same learning rate schedules and performed full (fp32) precision.

B. Performance Analysis

In the performance analysis part, four distributed training approaches are involved in our experiments: (1) *HIPPLE*: The *HIPPLE* with four stages and with two stages within single

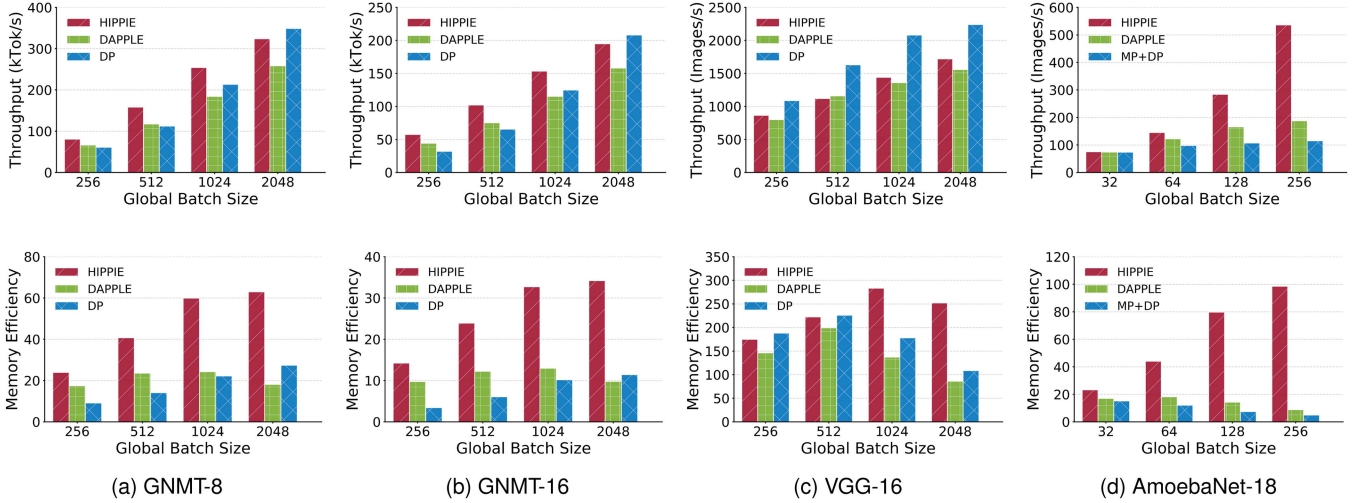


Fig. 5. Performance comparison of distributed training approaches using 16 GPUs.

nodes performs better as efficiency will be reduced greatly by the cross-node pipeline. (2) *DAPPLE*: Since *Gpipe* does not support data parallelism and there are 16 GPUs, models must be divided into 16 parts, which is inoperable. We had to implement a training process that integrates *Gpipe* and *DP* without applying checkpoints and hiding communication. Overall, there are four stages in *Gpipe* for our experiments. (3) *DP*: Data parallelism with intra-iteration computation communication overlap, which is one of the most efficient distributed training approaches under the PyTorch framework. (4) *MP+DP*: The method implemented to integrate *MP* and *DP* for training larger models.

Fig. 5 shows performance for all models with a fixed number of GPUs = 16. *HIPPIE* maintains a decent throughput. Meanwhile, as shown in the first column of Fig. 5, *HIPPIE* could keep the memory footprint to a sufficiently low value for each GBS configuration, only half of *DP* in some cases. Although sometimes checkpoints are at the expense of reducing throughput, the memory-efficiency index, one of our major optimization goals, is significantly improved. Moreover, the memory-efficiency index of *HIPPIE* is always the highest among the above three methods. We will present details in different models below.

GNMT16/GNMT8: Due to the existence of embedding layers, GNMT models hold a large number of parameters, and the computation cost of the LSTM layer is relatively small. Thereby, the proportion of communication overhead of GNMT is the highest among all models in our experiments. The throughput of *HIPPIE* exceeded data parallelism by 32%, 42%, 19% in GNMT-8, and 80%, 55%, 23% in GNMT-16, corresponding respectively to 256, 512, 1024 GBS. As for GBS 2048, along with the longer computation time, more communication overhead can be hidden via overlap in *DP*. Thus throughput of *DP* is slightly higher than *HIPPIE*. However, the peak memory overhead is reduced significantly by *HIPPIE* in this scenario. Furthermore, the memory footprint maintained by *HIPPIE* at 50%-60% of data parallelism regardless of the GBS. For less memory overhead approach *DAPPLE*, *HIPPIE* still drops memory footprint by around 35%. Altogether, when GBS ranged from 256 to 2048, *HIPPIE* could

get up to $2.86\times$ memory-efficiency index in GNMT-8, and $4.18\times$ in GNMT-16, compared to *DP*. Due to the fact that there is no scheme to hide the communication and optimize the memory overhead in *DAPPLE*, its memory-efficiency index falls in between *HIPPIE* and *DP*.

VGG-16: The model size of VGG-16 is slightly smaller than that of GNMT-16, while the convolutional layers take longer computation time, so the proportion of communication overhead is significantly reduced compared with GNMT. In VGG-16, the convolutional layers have a high computation cost and a small communication cost, while the fully connected layers are the opposite. Therefore, most communication operations could be covered through *DP* overlap during training VGG-16, resulting in high throughput. But *DP* consumes $1.17\times$, $1.43\times$, $2.3\times$ and $3.02\times$ peak memory, and finally achieves $1.07\times$, $1.01\times$, $0.62\times$ and $0.43\times$ memory-efficiency index respectively compared to *HIPPIE*. The performance of *DAPPLE* approach is worst in both memory overhead and throughput. The memory-efficiency index of *HIPPIE* is up to 192% better than *DAPPLE*.

AmoebaNet-18: Limited by the device memory, *DP* is unavailable in AmoebaNet. We have to divide AmoebaNet into several parts and then combine *Gpipe* or *MP* with *DP* for training. As Fig. 5 shows, *HIPPIE* performs significantly better than the other two approaches. Since *DAPPLE* does not follow the 1F1B principle [12], the memory overhead greatly limits the number of micro-batches and throughput. Meanwhile, with the help of a checkpoint, the memory overhead of *DAPPLE* is slightly lower than that of *MP+DP*. Although the *DAPPLE* approach has outperformed *MP+DP*, *HIPPIE* can decrease memory overhead by 25%-74% meanwhile increasing throughput by 10%-185%. Surprisingly, *HIPPIE* achieves up to $11\times$ and $20\times$ memory-efficiency index, respectively, compared to *DAPPLE* and *MP+DP*. This improvement enables *HIPPIE* to perform a pipeline with 2stages-8DP, which is more efficient than the pipeline with 4stages-4DP in *DAPPLE*. And the Communication Schedule further improves *HIPPIE*'s throughput and memory-efficiency index. It can be seen that *HIPPIE* has more

TABLE II
ADAPTIVE PLANNING RESULTS OF OUR PLANNER ON 16 GPUS

Model	GBS	Stages $\times DP$	N	Split Position	R
GNMT-8	256	4 $\times$ 4	8	2:3:4:3	Attention
	512	4 $\times$ 4	8	3:4:3:2	Attention
	1024	4 $\times$ 4	8	3:4:3:2	Attention
	2048	4 $\times$ 4	8	3:4:3:2	Attention
	4096	4 $\times$ 4	8	4:3:3:2	Attention
GNMT-16	256	4 $\times$ 4	8	3:6:6:5	Attention
	512	4 $\times$ 4	8	4:6:5:5	Attention
	1024	4 $\times$ 4	8	4:6:5:5	Attention
	2048	4 $\times$ 4	8	5:6:5:4	Attention
	4096	4 $\times$ 4	8	5:6:4:5	Attention
VGG-16	Any	2 $\times$ 8	8	6:15	None
AmoebaNet-18	Any	2 $\times$ 8	4	9:15	None

Models are configured with different global batch sizes (GBS), parallel strategies (Stages $\times DP$), and micro-batch sizes (N). The results include the number of model layers contained in each stage (Split Position) and layers that need to be recomputed (R).

outstanding training efficiency for large-scale models, especially those that cannot be trained by a single GPU. *HIPPIE* can limit the maximum memory consumption of AmoebaNet training to 6 GB with an impressive speed, allowing us to arrange the training in wimpy GPUs such as 1080Ti, greatly reducing the research cost.

C. Planning Results Analysis

We first analyze the performance of the *HIPPIE* planner, mainly from the perspective of model division. We compare the training performance of *HIPPIE* under different model division strategies to verify the advantages of the *HIPPIE* planner strategy. Altogether, three model partition methods are involved: (1) *HIPPIE*: The method discussed in Section IV-B. It divides the model with Memory Efficiency as the optimization goal and automatically gives the layer to which the checkpoint is applied. (2) *Layer*: The method divides the model evenly according to the number of model layers without using an activation checkpoint. (3) *Weight*: This method divides the model evenly according to the parameters of the model layer and does not apply activation checkpointing. The current popular hybrid parallel training frameworks, such as DeepSpeed [35], mainly use the Layers and Weight methods mentioned above. For a fair comparison, all partition methods use *HIPPIE* for training after the model partitioning. Therefore, the difference in results was not significant.

Table II shows the adaptive planning results of *HIPPIE Planner* with the configuration of 16GPUs. The table contains settings for five aspects. The first is the global batch size (GBS). Different GBS settings will cause changes in the activation size, computation time, and division position. Second, $x(Stages) \times y(DP)$ means that the model is divided into x parts to form a x -stage pipeline, and there are y pipelines to

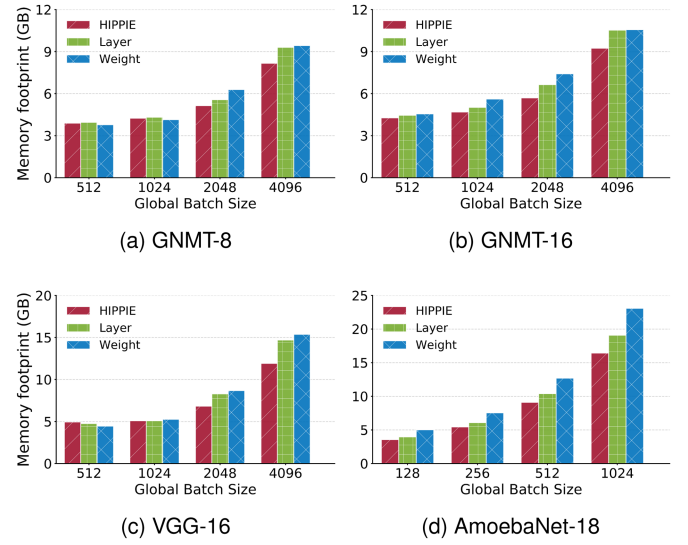


Fig. 6. Memory footprint with model partition schemes, the lower, the better.

form the data parallelism. The division position corresponds to the number of model layers in each stage, whose summation is the total number of model layers. N refers to the number of micro-batches, and R refers to the layer to which checkpoint needs to be applied.

The GNMT model has more stages and, thus, more partition options. As shown in the table, the split position changes with GBS. As GBS changes from small to large, *HIPPIE* planner allocates more layers to the first half of the pipeline. Since the increment in batch size will have different effects on different types of layers. For the Attention layers located at the back of the model, the memory overhead and computation time increase more significantly, and so does the fully connected layer. Therefore, *HIPPIE* planner could dynamically adjust the model partition scheme, an advantage the other two methods do not have. In addition, *HIPPIE* planner applies a checkpoint to the Attention layer to improve the memory-efficiency index.

Memory overhead: Fig. 6 shows the memory overhead comparison of the three model partitioning methods. When the GBS is small, the model parameter data accounts for a more significant proportion of the memory overhead. There is little difference between the three methods. However, when the GBS is getting larger, the proportion of activation increases as well. *HIPPIE* has considered the activation when planning, so the memory overhead is significantly lower than the other two methods. The model parameters for the GNMT-8 and GNMT-16 models take a relatively large proportion of the memory overhead. Furthermore, the model layer is mainly composed of the LSTM layers. The Layer method and the Weight method can also achieve acceptable memory overhead. For the VGG-16 model and the AmoebaNet model, the activation during training accounts for a more significant proportion, so the advantages of the *HIPPIE* method are more evident than others. Corresponding to these four models, *HIPPIE* can save up to 12.2%, 14.3%, 18.8%, and 13.9% of the memory overhead compared to the

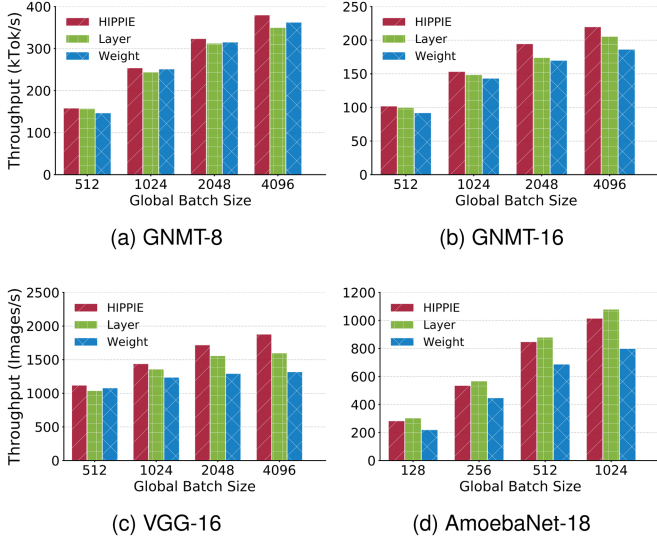


Fig. 7. Throughput with model partition schemes, the higher the better.

Layer method, respectively. Compared with the Weight method, it can save up to 18.2%, 23.2%, 22.5%, and 28.8%, respectively.

Throughput: Fig. 7 shows the throughput comparison of training using each model partition scheme. In the pipeline, the balance of each stage is crucial. The imbalance of the pipeline will not only increase the total computation time but also make the communication overhead more prominent. For GNMT-8 and GNMT-16, *HIPPIE* is slightly higher than the other two methods. *HIPPIE* takes into account the memory overhead, and the checkpoint will slightly reduce the throughput. It improves the throughput by up to 8.5% and 11.8%, compared with the Layers method, and up to 7.7% and 18.0%, compared with the Weight method, respectively. For the CNN model, the parameter distribution is very imbalanced. More than 80% of the parameters are distributed in the final fully connected layer in VGG-16. Therefore, the parameter balance scheme provided by the Weight method is 18:3. At the same time, the characteristic of the VGG-16 model is that the computation time of the convolutional layer at the front is longer. In comparison, the computation time of the fully connected layer at the rear is shorter. This model partition scheme is very imbalanced, resulting in low training throughput. The Layers method makes the number of layers in each stage the same, and the imbalance problem is improved, but the throughput is still lower than *HIPPIE*. The *HIPPIE* method improved throughput by up to 17.5% compared with the Layers method and 42.5% compared with the Weight method. For Amoebanet-18, similar to VGG-16, the Weight method still achieves the lowest throughput. However, *HIPPIE* obtains a slightly lower throughput than the Layers method. Due to the significant memory overhead during Amoebanet-18 training, *HIPPIE* sacrifices part of the throughput in the tradeoff between memory overhead and throughput to achieve a greater memory-efficiency index. *HIPPIE* improves throughput by up to 29.0% compared to the Weight method.

Memory Efficiency: We next quantify the memory-efficiency index. The *HIPPIE* method clearly shows its advantages. As

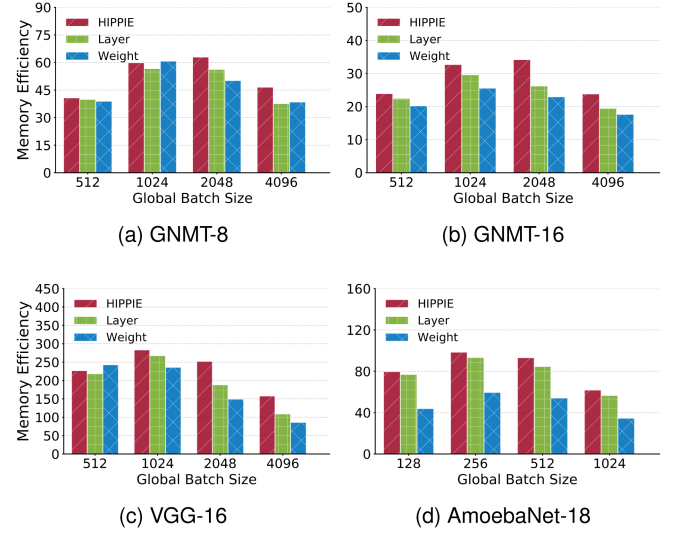


Fig. 8. Memory efficiency with model partition schemes, the higher, the better.

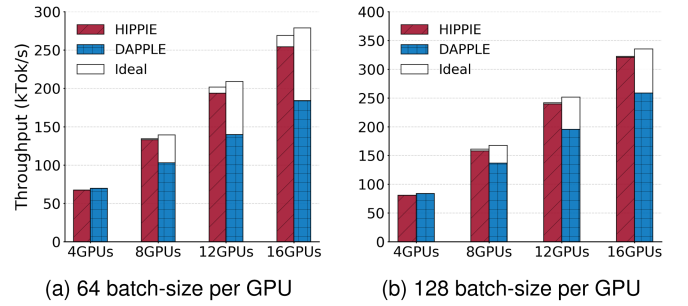


Fig. 9. Multi-GPU scaling performance for GNMT-8.

shown in Fig. 8, the *HIPPIE* method has achieved the highest memory-efficiency index under common training settings. In the training of GNMT-8, GNMT-16, VGG-16, and Amoebanet-18, *HIPPIE* outperforms the Layer method by up to 23.6%, 30.5%, 44.8%, and 10.1%. Compared with the Weight method, the memory-efficiency index can be improved by 25.5%, 49%, 83.8%, and 81.5%, respectively. As the increasing of batch size, the activation during training gradually dominates. *HIPPIE* dynamically considers various factors in the training process. At the same time, the other two methods only divide the model based on static data, such as the number of layers or parameters, and cannot give the optimal model division plan.

D. Scalability

This section compares scalability with a fixed batch size per GPU. The GNMT, which has the highest proportion of communication overhead, is selected for this experiment. We compare the scalability of *HIPPIE* and *DAPPLE* on 4, 8, 12, and 16 GPUs. The average batch size of each GPU is set to 64 and 128, commonly used in GNMT training. We use the throughput of 4 GPUs as the benchmark, then the throughput of 16 GPUs in linear scaling should be four times. Figs. 9 and 10 show the

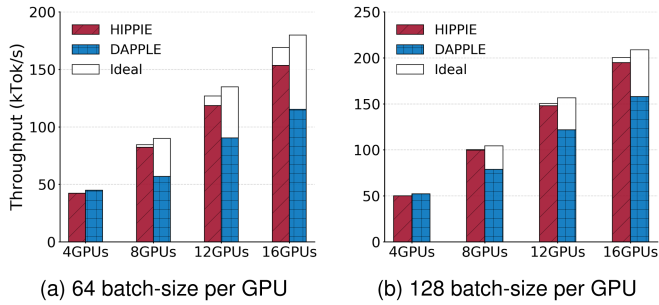


Fig. 10. Multi-GPU scaling performance for GNMT-16.

throughput of both approaches and compare them to ideal linear scaling results. Due to space, we did not show the scalability chart of other models. VGG-16 and Amoebanet-18 have better scalability because of the lower proportion of communication.

Multi-GPU scaling performance for GNMT-8 is shown in Fig. 9. With 64 batch-size, scaled from 4 GPUs to 16 GPUs, *HIPPIE* gets up to $3.78\times$ throughput while *DAPPLE* gets $2.64\times$. With 128 batch-size, the improvement is $3.98\times$ in *HIPPIE* while $3.08\times$ in *DAPPLE*. For GNMT-16 shown in Fig. 10, when scaled to 16GPUs, *HIPPIE* gets $3.63\times$ throughput while *DAPPLE* gets $2.56\times$ with 64 batch-size. And *HIPPIE* gets $3.88\times$ throughput while *DAPPLE* gets $3.02\times$ with 128 batch-size.

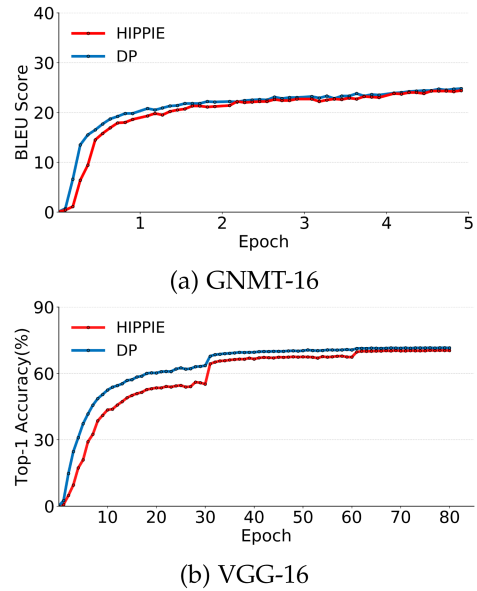
Communication Schedule is proved to bring a significant increase in scalability, which makes *HIPPIE* outperform other hybrid parallel approaches vastly. With the increasing number of GPUs, *HIPPIE*'s robust scalability could be more prominent. On the other hand, unlike most hybrid parallel methods, *HIPPIE* only needs to copy the same pipeline to more nodes after determining the model partition and pipeline scheme, making the implementation more convenient.

E. Convergence

The convergence of *HIPPIE* training DNN models is shown in Fig. 11. We traced the BLEU score for GNMT-16 and accuracy values for VGG-16. From our observation, *HIPPIE* can reach target accuracy in a closing number of epochs to *DP*, confirming that Communication Schedule in Section V-A would not affect the convergence.

VIII. RELATED WORK

Data parallelism (DP): Optimizations on *DP* have been extensively studied. ZeRO [36] of DeepSpeed [35] eliminates memory redundancy by partitioning the model states among all workers. ZeRO-Infinity [37] and PatrickStar [38] swap memory to the CPU and NVMe. The memory optimization in *DP* is orthogonal to *HIPPIE* and can potentially further improve memory efficiency. Several communication optimization approaches [39], [40] are developed: Decentralized collective operations [41], [42], [43] improve communication efficiency [8], [44], [45]. Asynchronous Parallel (ASP) [46] and Stale Synchronous Parallel (SSP) [47], [48] allow asynchronous updates and relax the restriction on data dependency. Inspired

Fig. 11. Accuracy versus epoch training with *HIPPIE* on 16 GPUs.

by the previous asynchronous approaches, *HIPPIE* employs a synchronous-asynchronous mixed updates methods.

Model parallelism (MP): Alex Krizhevsky [49] manually partitioned AlexNet, applied the *MP* to linear layers with many parameters but few outputs, and *DP* to convolutional layers with few parameters but large output, which is an early attempt at hybrid parallelism. FlexFlow [50] proposed a new search space SOAP for DNN parallelization strategy to seek the optimal hybrid scheme for *MP* and *DP* from four dimensions: sample, operator, attribute, and parameter. Due to the distinct layers in models, the single parallel method cannot achieve optimal performance, and hybrid parallelism is a whole new ball game [51], [52].

Pipeline model parallelism (PMP): With the introduction of *PMP*, researchers have integrated *PMP* and *DP*. HetPipe [53] combined parameter server with *PMP* and explored hybrid parallelism under heterogeneous architecture to improve the efficiency of DNN training. The pipeline and parameter server parts of Hetpipe both adopted the idea of asynchronous updates, but too much stale data will inevitably affect the convergence of the model. Multiple versions of model parameters stored in memory increase memory overhead as well. Alpa [54] presents a fine-grained parallelism scheme with automated inter- and intra-operators. Pipetransformer [55] uses an automated elastic pipelining for large-scale models. For *PMP* scheduling optimizations, Chimera [56] uses a bidirectional pipeline to train large models, Fold3D [57] schedules the communication operations of *PMP*, and AvgPipe [58] increase the number of microbatches to improve the device efficiency. These methods can be integrated with *HIPPIE* to further eliminate the pipeline bubbles. Similar to *DP* and *MP*, the gradient synchronization overhead problem persists, reducing the training scalability, and the lack of a model partitioning plan complicates the distributed training. And these are the core problems that *HIPPIE* tries to solve.

IX. CONCLUSION

In this paper, we present a distributed training framework for large-scale DNN models, *HIPPIE*, which integrates pipelined model parallelism with data parallelism. To address the challenges of large DNN training, *HIPPIE* introduces the Communication Schedule to reduce communication overhead and improve the scalability in large-scale computing nodes. In addition, the *HIPPIE Planner* significantly optimizes the process of model partition and eases the difficulties of distributed training. Experimental results show that Communication Schedule enables *HIPPIE* to maintain 90% scaling efficiency on a 16-GPU platform. The *HIPPIE Planner* improves the memory-efficiency index by 25% to 80% compared with other model partitioning methods. Moreover, *HIPPIE* outperforms *DP* by up to $4.18\times$ memory-efficiency index.

REFERENCES

- [1] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," pp. 1–12, 2018.
- [2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019. [Online]. Available: <https://github.com/openai/gpt-2>
- [3] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [4] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for youtube recommendations," in *Proc. 10th ACM Conf. Recommender Syst.*, 2016, pp. 191–198.
- [5] J. Wang, P. Huang, H. Zhao, Z. Zhang, B. Zhao, and D. L. Lee, "Billion-scale commodity embedding for e-commerce recommendation in alibaba," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 839–848.
- [6] M. Li et al., "Scaling distributed machine learning with the parameter server," in *Proc. 11th USENIX Symp. Operating Syst. Des. Implementation*, Broomfield, CO, USENIX Association, 2014, pp. 583–598.
- [7] W. Dai, A. Kumar, J. Wei, Q. Ho, G. Gibson, and E. Xing, "High-performance distributed ML at scale through parameter server consistency models," in *Proc. AAAI Conf. Artif. Intell.*, 2015, pp. 79–87.
- [8] A. Sergeev and M. D. Balso, "Horovod: Fast and easy distributed deep learning in tensorflow," 2018, *arXiv: 1802.05799*. [Online]. Available: <http://arxiv.org/abs/1802.05799>
- [9] J. K. Kim et al., "STRADS: A distributed framework for scheduled model parallel machine learning," in *Proc. 11th Eur. Conf. Comput. Syst.*, New York, NY, USA, 2016, pp. 1–16. [Online]. Available: <https://doi.org/10.1145/2901318.2901331>
- [10] J. Dean et al., "Large scale distributed deep networks," in *Proc. 25th Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 1223–1231.
- [11] X. Ye et al., "Hippie: A data-parallelized pipeline approach to improve memory-efficiency and scalability for large DNN training," in *Proc. 50th Int. Conf. Parallel Process.*, Lemont, IL, USA, 2021, pp. 71:1–71:10. [Online]. Available: <https://doi.org/10.1145/3472456.3472497>
- [12] D. Narayanan et al., "Pipedream: Generalized pipeline parallelism for DNN training," in *Proc. 27th ACM Symp. Operating Syst. Princ.*, New York, NY, USA, 2019, pp. 1–15. [Online]. Available: <https://doi.org/10.1145/3341301.3359646>
- [13] Y. Huang et al., "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 103–112.
- [14] J. Zhan and J. Zhang, "Pipe-torch: Pipeline-based distributed deep learning in a GPU cluster with heterogeneous networking," in *Proc. 7th Int. Conf. Adv. Cloud Big Data*, 2019, pp. 55–60.
- [15] B. Yang, J. Zhang, J. Li, C. Ré, C. Aberger, and C. De Sa, "PipeMare: Asynchronous pipeline parallel DNN training," in *Proc. Mach. Learn. Syst.*, 2021, vol. 3, pp. 269–296.
- [16] C. Oh, Z. Zheng, X. Shen, J. Zhai, and Y. Yi, "GOPipe: A granularity-oblivious programming framework for pipelined stencil executions on GPU," in *Proc. ACM Int. Conf. Parallel Architectures Compilation Techn.*, 2020, pp. 43–54.
- [17] Z. Zheng, C. Oh, J. Zhai, X. Shen, Y. Yi, and W. Chen, "VersaPipe: A versatile programming framework for pipelined computing on GPU," in *Proc. IEEE/ACM 50th Annu. Int. Symp. Microarchitecture*, 2017, pp. 587–599.
- [18] S. Lee, J. K. Kim, X. Zheng, Q. Ho, G. Gibson, and E. P. Xing, "On model parallelization and scheduling strategies for distributed machine learning," in *Proc. 27th Int. Conf. Neural Inf. Process. Syst.*, 2014, pp. 2834–2842.
- [19] C.-C. Chen, C.-L. Yang, and H.-Y. Cheng, "Efficient and robust parallel DNN training through model parallelism on multi-GPU platform," 2018, *arXiv: 1809.02839*.
- [20] D. Narayanan, A. Phanishayee, K. Shi, X. Chen, and M. Zaharia, "Memory-efficient pipeline-parallel DNN training," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 7937–7947.
- [21] S. Fan et al., "DAPPLE: A pipelined data parallel approach for training large models," in *Proc. 26th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, 2021, pp. 431–445.
- [22] Z. Lai et al., "Merak: An efficient distributed DNN training framework with automated 3D parallelism for giant foundation models," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 5, pp. 1466–1478, May 2023.
- [23] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," 2016, *arXiv:1604.06174*. [Online]. Available: <http://arxiv.org/abs/1604.06174>
- [24] P. Jain et al., "Checkmate: Breaking the memory wall with optimal tensor rematerialization," in *Proc. Mach. Learn. Syst.*, 2020, vol. 2, pp. 497–511.
- [25] D. Narayanan et al., "Efficient large-scale language model training on GPU clusters using megatron-LM," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2021, pp. 1–15.
- [26] W. Liu, Z. Lai, S. Li, Y. Duan, K. Ge, and D. Li, "AutoPipe: A fast pipeline parallelism approach with balanced partitioning and micro-batch slicing," in *Proc. IEEE Int. Conf. Cluster Comput.*, 2022, pp. 301–312.
- [27] O. Beaumont, L. Eyraud-Dubois, and A. Shilova, "Pipelined model parallelism: Complexity results and memory considerations," in *Proc. Parallel Process. 27th Int. Conf. Parallel Distrib. Comput.*, Lisbon, Portugal, Springer, 2021, pp. 183–198. [Online]. Available: https://doi.org/10.1007/978-3-030-85665-6_12
- [28] T. Sun, R. Hannah, and W. Yin, "Asynchronous coordinate descent under more realistic assumptions," in *Proc. Adv. Neural Inf. Process. Syst.*, Curran Associates, Inc., 2017, pp. 6183–6191. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/file/fb2606a5068901da92473666256e6e5b-Paper.pdf>
- [29] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [30] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4780–4789.
- [31] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.
- [32] Y. Wu et al., "Google's neural machine translation system: Bridging the gap between human and machine translation," 2016, *arXiv:1609.08144*.
- [33] R. Sennrich, B. Haddow, and A. Birch, "Edinburgh neural machine translation systems for WMT 16," 2016, *arXiv:1606.02891*.
- [34] "NCCL," 2019. [Online]. Available: <https://developer.nvidia.com/nccl>
- [35] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA, 2020, pp. 3505–3506. [Online]. Available: <https://doi.org/10.1145/3394486.3406703>
- [36] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2020, pp. 1–16.
- [37] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2021, pp. 1–14.
- [38] J. Fang et al., "Parallel training of pre-trained models via chunk-based dynamic memory management," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 1, pp. 304–315, Jan. 2023.
- [39] A. V. Gerbessiotis and L. G. Valiant, "Direct bulk-synchronous parallel algorithms," *J. Parallel Distrib. Comput.*, vol. 22, no. 2, pp. 251–267, 1994.
- [40] X. Meng et al., "MLlib: Machine learning in apache spark," *J. Mach. Learn. Res.*, vol. 17, no. 1, pp. 1235–1241, 2016.
- [41] P. Goyal et al., "Accurate, large minibatch SGD: Training imagenet in 1 hour," 2017, *arXiv: 1706.02677*.

- [42] X. Lian, W. Zhang, C. Zhang, and J. Liu, "Asynchronous decentralized parallel stochastic gradient descent," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 3043–3052.
- [43] Y. Li, M. Yu, S. Li, S. Avestimehr, N. S. Kim, and A. Schwing, "Pipe-SGD: A decentralized pipelined SGD framework for distributed deep net training," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 8056–8067.
- [44] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, vol. 32, pp. 8024–8035. [Online]. Available: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>
- [45] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," in *Proc. 12th USENIX Conf. Operating Syst. Des. Implementation*, 2015, pp. 265–283.
- [46] B. Recht, C. Re, S. Wright, and F. Niu, "HOGWILD!: A lock-free approach to parallelizing stochastic gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, 2011, pp. 693–701.
- [47] Q. Ho et al., "More effective distributed ML via a stale synchronous parallel parameter server," in *Proc. Adv. Neural Inf. Process. Syst.*, 2013, pp. 1223–1231.
- [48] H. Cui et al., "Exploiting bounded staleness to speed up Big Data analytics," in *Proc. USENIX Annu. Tech. Conf.*, 2014, pp. 37–48.
- [49] A. Krizhevsky, "One weird trick for parallelizing convolutional neural networks," 2014, *arXiv:1404.5997*. [Online]. Available: <http://arxiv.org/abs/1404.5997>
- [50] Z. Jia, M. Zaharia, and A. Aiken, "Beyond data and model parallelism for deep neural networks," in *Proc. Mach. Learn. Syst.*, 2019, pp. 1–13.
- [51] T. Chilimbi, Y. Suzue, J. Apacible, and K. Kalyanaraman, "Project adam: Building an efficient and scalable deep learning training system," in *Proc. 11th USENIX Symp. Operating Syst. Des. Implementation*, 2014, pp. 571–582.
- [52] S. Pal et al., "Optimizing multi-GPU parallelization strategies for deep learning training," *IEEE Micro*, vol. 39, no. 5, pp. 91–101, Sep./Oct. 2019.
- [53] J. H. Park et al., "HetPipe: Enabling large DNN training on (whimpy) heterogeneous GPU clusters through integration of pipelined model parallelism and data parallelism," in *Proc. USENIX Annu. Tech. Conf.*, 2020, pp. 307–321.
- [54] L. Zheng et al., "Alpa: Automating inter-and intra-operator parallelism for distributed deep learning," in *Proc. 16th USENIX Symp. Operating Syst. Des. Implementation*, 2022, pp. 559–578.
- [55] C. He, S. Li, M. Soltanolkotabi, and S. Avestimehr, "PipeTransformer: Automated elastic pipelining for distributed training of large-scale models," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 4150–4159.
- [56] S. Li and T. Hoefler, "Chimera: Efficiently training large-scale neural networks with bidirectional pipelines," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, New York, NY, USA, 2021, pp. 1–14. [Online]. Available: <https://doi.org/10.1145/3458817.3476145>
- [57] F. Li et al., "Fold3D: Rethinking and parallelizing computational and communicational tasks in the training of large DNN models," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 5, pp. 1432–1449, May 2023.
- [58] Z. Chen, C. Xu, W. Qian, and A. Zhou, "Elastic averaging for efficient pipelined DNN training," in *Proc. 28th ACM SIGPLAN Annu. Symp. Princ. Pract. Parallel Program.*, 2023, pp. 380–391.



Dongsheng Li received the PhD degree in computer science and technology from National University of Defense Technology (NUDT), in 2005. He is a professor and doctoral supervisor with the College of Computer, National University of Defense Technology (NUDT). He was awarded the Chinese National Excellent Doctoral Dissertation in 2008. His research interests include distributed systems, cloud computing, and Big Data processing.



Shengwei Li received the BS degree from Nanjing University, Jiangsu, China, in 2017, and the MS degree in computer science from Stony Brook University, New York, USA, in 2020. He is currently working toward the PhD degree with the College of Computer, National University of Defense Technology (NUDT). His research interests include high-performance computing and distributed machine learning systems.



Zhiquan Lai received the BS, MS, and PhD degrees in computer science from the National University of Defense Technology (NUDT), in 2008, 2010, and 2015, respectively. He is currently an associate professor with the National Key Laboratory of Parallel and Distributed Computing of NUDT. He worked as a research assistant with the Department of Computer Science, the University of Hong Kong during 2012 to 2013. His current research interests include high-performance system software, distributed machine learning, and power-aware computing.



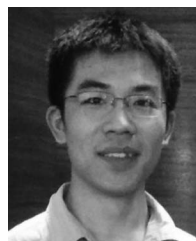
Yongquan Fu is currently an associate professor with the National University of Defense Technology, China. His current research interests include network computing, network measurement, and distributed systems.



Xiangyu Ye received the bachelor's degree in computer science from Nanjing University, China, in 2019, and the MS degree from the College of Computer, National University of Defense Technology (NUDT), in 2021. His current interests are mainly in optimization techniques related to large-scale model training.



Lei Cai received the bachelor's degree in information engineering from the National University of Defense Technology (NUDT), in 2009, and the MS degree from NUDT, in 2011. His current interests are mainly in optimization techniques related to large-scale model training.



Linbo Qiao received the BS, MS, and PhD degrees in computer science and technology from the National University of Defense Technology (NUDT), Changsha, China, in 2010, 2012, and 2017, respectively. He is currently an associate professor with the National Key Laboratory of Parallel and Distributed Computing, NUDT. He worked as a research assistant with the Chinese University of Hong Kong, from May 2014 to Oct. 2014. His research interests include structured sparse learning, online and distributed optimization, and deep learning for graph and graphical models.